#1 Packed vs Unpacked Array

- **목표**: packed/unpacked array 차이, slicing, concatenation 이해.
- **아래 코드를 보고 TODO를 채워 넣어 packed/unpacked array의 차이를 확인**

```
module arr_test;

  // TODO 1: packed array 선언 (8비트)
  logic [7:0] p_arr = 8'hA5;

  // TODO 2: unpacked array 선언 (8개 원소, 각 원소는 1비트)
  logic u_arr[8];

  initial begin
    // TODO 3: p_arr의 각 비트를 unpacked 배열에 복사
    // for문 사용
    for (int i = 0; i < 8; i++) begin
      u_arr[i] = p_arr[7 - i];
    end

    // TODO 4: p_arr[3:0] slicing 출력
    $display("Lower nibble = %b", p_arr[3:0]);

    // TODO 5: u_arr을 { } concatenation으로 packed 형태로 출력
    $display("u_arr as packed = %b", {u_arr[0],u_arr[1],u_arr[2],u_arr[3],
                                      u_arr[4],u_arr[5],u_arr[6],u_arr[7]});
  end

endmodule
```

#1 Packed vs Unpacked Array

```
module arr_test;

  // packed 8-bit array
  logic [7:0] p_arr = 8'hA5;

  // unpacked array (8 elements, 1 bit each)
  logic u_arr[8];

  initial begin
    // p_arr → u_arr 복사
    for (int i = 0; i < 8; i++) begin
      u_arr[i] = p_arr[7 - i];
    end

    // slicing
    $display("Lower nibble = %b", p_arr[3:0]);

    // concatenation 출력
    $display("u_arr as packed = %b",
             {u_arr[0],u_arr[1],u_arr[2],u_arr[3],
              u_arr[4],u_arr[5],u_arr[6],u_arr[7]});
  end

endmodule
```

#2 Operators & Operands 실습

- **목표:** SystemVerilog 연산자 우선순위 및 논리/비트 연산을 직접 테스트한다.
- **문제 설명:** 아래 코드를 완성하여 연산 결과가 어떻게 달라지는지 비교하라.

```
module op_test;

  logic [3:0] a = 4'b1101;
  logic [3:0] b = 4'b0110;

  initial begin
    // TODO 1: AND, OR, XOR 결과 출력
    $display("a & b = %b", a & b);
    $display("a | b = %b", a | b);
    $display("a ^ b = %b", a ^ b);

    // TODO 2: 논리 연산 &&, || 적용 결과 출력
    $display("(a && b) = %0d", a && b);
    $display("(a || b) = %0d", a || b);

    // TODO 3: 연산자 우선순위 비교
    $display("a + b << 1 = %b", (a + b) << 1);
  end

endmodule
```

#3 Event scheduling & Race condition

- 목표: SystemVerilog 이벤트 큐, 블로킹/논블로킹, race condition의 이해.
- 문제: 아래 코드는 race condition을 유발한다. race가 발생하는 이유를 설명하고 코드를 수정하라.

```
module race_test;

  logic clk = 0;
  logic x, y;

  always #5 clk = ~clk;

  // 원래 코드(문제 포함)
  always @(posedge clk) x = y;
  always @(posedge clk) y = x;

  // TODO: race condition 제거 위해 수정
  // 힌트: nonblocking assignment 또는 한 블록으로 합치기

  initial begin
    x = 0; y = 1;
    #20 $finish;
  end

endmodule
```

#3 Event scheduling & Race condition(answer)

- 문제 코드의 race는 blocking assignment가 두 블록에서 동시에 posedge에 실행되기 때문에 발생함.
- 해결: nonblocking 사용 또는 한 always로 합치기.

```
module race_test;

  logic clk = 0;
  logic x, y;

  always #5 clk = ~clk;

  // Nonblocking으로 race 제거
  always @(posedge clk) begin
    x <= y;
    y <= x;
  end

  initial begin
    x = 0; y = 1;
    #20 $finish;
  end

endmodule
```

#4 Mini Pipelined ALU with Event Scheduling and Array Storage

- **목표: 2일차 개념을 모두 활용하여 실습**
  - Packed/unpacked array
  - Operators
  - Event scheduling
  - Nonblocking assignment
  - Simple pipeline stage 구성
- **문제:** 다음 요구사항을 만족하는 2-stage 파이프라인 ALU를 구현하라.
  1. 입력 데이터는 unpacked array 형태로 4개의 연산 요청이 들어온다.

```
typedef struct {
  logic [3:0] op;    // 0:add, 1:sub, 2:and, 3:or
  logic [7:0] a;
  logic [7:0] b;
} alu_req_t;

alu_req_t req_q[4];   // unpacked array
```

  2. stage1: 입력 요청을 읽어서 연산 준비 (blocking or nonblocking?)
  3. stage2: 실제 연산을 수행하고 결과는 packed array 형태로 저장

```
logic [7:0] result_packed[4];  // packed array 저장
```

#4 Mini Pipelined ALU with Event Scheduling and Array Storage

- **목표: 2일차 개념을 모두 활용하여 실습**
  - Packed/unpacked array
  - Operators
  - Event scheduling
  - Nonblocking assignment
  - Simple pipeline stage 구성
- **문제:** 다음 요구사항을 만족하는 2-stage 파이프라인 ALU를 구현하라.
  1. 입력 데이터는 unpacked array 형태로 4개의 연산 요청이 들어온다.

```
typedef struct {
  logic [3:0] op;    // 0:add, 1:sub, 2:and, 3:or
  logic [7:0] a;
  logic [7:0] b;
} alu_req_t;

alu_req_t req_q[4];   // unpacked array
```

  2. stage1: 입력 요청을 읽어서 연산 준비 (blocking or nonblocking?)
  3. stage2: 실제 연산을 수행하고 결과는 packed array 형태로 저장

```
logic [7:0] result_packed[4];  // packed array 저장
```

#4 Mini Pipelined ALU with Event Scheduling and Array Storage

- **문제:** 다음 요구사항을 만족하는 2-stage 파이프라인 ALU를 구현하라.
  4. event scheduling이 의도대로 동작하도록 반드시 nonblocking 사용
  5. 모든 4개 요청 처리 후 result_packed를 하나씩 출력

```
module pipeline_alu;

  typedef struct {
    logic [3:0] op;
    logic [7:0] a;
    logic [7:0] b;
  } alu_req_t;

  alu_req_t req_q[4];
  logic [7:0] stage1_a, stage1_b;
  logic [3:0] stage1_op;
  logic [7:0] result_packed[4];

  logic clk = 0;
  always #5 clk = ~clk;

  initial begin
    // TODO: 4개 요청 초기화
    req_q[0] = '{0, 8'd10, 8'd3};
    req_q[1] = '{1, 8'd20, 8'd5};
    req_q[2] = '{2, 8'd7,  8'd12};
    req_q[3] = '{3, 8'd8,  8'd2};
  end
```

```
  // TODO: stage1 - req_q[i] 읽어서 stage1 레지스터에
저장
  always @(posedge clk) begin
    // i 증가 로직 포함
  end
  // TODO: stage2 - stage1 결과 계산 후 result 저장
  always @(posedge clk) begin
    case(stage1_op)
      0: result_packed[i] <= stage1_a + stage1_b;
      1: result_packed[i] <= stage1_a - stage1_b;
      2: result_packed[i] <= stage1_a & stage1_b;
      3: result_packed[i] <= stage1_a | stage1_b;
    endcase
  end

  initial begin
    #200;
    for (int j = 0; j < 4; j++)
      $display("Result[%0d] = %0d", j,
result_packed[j]);
    $finish;
  end

endmodule
```

#4 Mini Pipelined ALU with Event Scheduling and Array Storage (answer)

```
module pipeline_alu;

  typedef struct {
    logic [3:0] op;
    logic [7:0] a;
    logic [7:0] b;
  } alu_req_t;

  alu_req_t req_q[4];

  logic [7:0] stage1_a, stage1_b;
  logic [3:0] stage1_op;
  logic [7:0] result_packed[4];

  int i = 0;   // pipeline index

  logic clk = 0;
  always #5 clk = ~clk;

  initial begin
    req_q[0] = '{0, 8'd10, 8'd3};
    req_q[1] = '{1, 8'd20, 8'd5};
    req_q[2] = '{2, 8'd7,  8'd12};
    req_q[3] = '{3, 8'd8,  8'd2};
  end
```

```
  // Stage1: 요청 읽기
  always @(posedge clk) begin
    if (i < 4) begin
      stage1_op <= req_q[i].op;
      stage1_a  <= req_q[i].a;
      stage1_b  <= req_q[i].b;
    end
  end
  // Stage2: 연산 수행
  always @(posedge clk) begin
    if (i < 4) begin
      case (stage1_op)
        0: result_packed[i] <= stage1_a + stage1_b;
        1: result_packed[i] <= stage1_a - stage1_b;
        2: result_packed[i] <= stage1_a & stage1_b;
        3: result_packed[i] <= stage1_a | stage1_b;
      endcase
      i++;
    end
  end
  initial begin
    #200;
    for (int j = 0; j < 4; j++)
      $display("Result[%0d] = %0d", j,
result_packed[j]);
    $finish;
  end
endmodule
```